

Object-Oriented Programming

Workshop 3

Application of SOLID Principles in the DistriCine Project

Authors:

Diego Alejandro Yañez Zabala
(20251020103)

Brayan Sebastian Diaz Ramirez
(20251020150)

Oscar Javier Camargo Trujillo
(20251020143)

Universidad Distrital Francisco José de Caldas
Faculty of Engineering
Systems Engineering Program

Teacher: Ing. Carlos Andrés Sierra, M.Sc

Bogotá D.C., Colombia
2025

1. Introduction

The management of cultural and recreational activities at the university, such as film clubs, has historically relied on manual processes that include going from classroom to classroom, posting flyers, and distributing leaflets to invite students to scheduled screenings. These methods are time-consuming, generate little visibility, and rarely capture the attention of the student community, resulting in inefficient resource allocation and low attendance due to a lack of planning. In response to these limitations, *DistriCine* was launched at the *Francisco José de Caldas District University*, a project aimed at digitizing, organizing, and optimizing the management of university film screenings across multiple campuses.

While commercial cinema management systems exist, they are unsuitable for academic settings: their complexity, high cost, and lack of integration with institutional functionalities (such as university identification, multi-site management, or simplified processes) make them impractical for a university film club. *DistriCine* therefore proposes a lightweight, specialized system fully adapted to the needs of the university.

The project adopts Object-Oriented Programming (OOP) as its fundamental paradigm. This acts as a "master script" that structures the system: encapsulation defines the specific responsibility of each class, inheritance establishes clear hierarchies between user types, and polymorphism allows different components to respond in specific ways to shared actions. The application of object-oriented programming was initially designed during Workshop 1 (Conceptual Design), and then implemented in Workshop 2 (more precisely, the application of OOP principles), where the system's main classes were built and the development of *DistriCine* began.

In this third stage, corresponding to Workshop 3, the project evolves from a basic object-oriented design to a robust and maintainable architecture through the explicit integration of SOLID principles. This workshop involves reviewing and improving the requirements, user stories, CRC cards, and the system's UML model, ensuring that the design adequately addresses criteria of extensibility, cohesion, loose coupling, and interchangeability.

2. Requirements Update

2.1. Updated Functional Requirements (FR)

Cuadro 1: Functional Requirements

ID	Requirement
FR1	The system must allow authentication via email and password.
FR2	The system must handle two roles: Administrator and Student .
FR3	Students should be able to consult the film catalog.
FR4	Students should be able to book performances.
FR5	Administrators must be able to manage movies.
FR6	Administrators must be able to manage screening schedules.
FR7	Administrators must be able to manage university campuses.
FR8	The system must show seat availability for each function.
FR9	The system must display a transactional flow in the console.
FR10	All modules must respect the Single Responsibility Principle (SRP).
FR11	The system must be extensible without modifying existing code (OCP).

2.2. Non-Functional Requirements (NFR)

Cuadro 2: Non-Functional Requirements

ID	Requirement
NFR1	Response time less than 3 seconds.
NFR2	Simple and clear console interface.
NFR3	Classes must comply with SRP to facilitate maintenance.
NFR4	The design must be open to extension and closed to modification (OCP).
NFR5	Dependencies should point to abstractions (DIP).
NFR6	Subclasses must be interchangeable without breaking behavior (LSP).
NFR7	Interfaces should be small and specific (ISP).

2.3. Updated User Stories

Cuadro 3: User Stories

ID	Role	Description
US1	Student	As a student, I want to see the list of movies to choose a show.
US2	Student	As a student, I want to reserve a performance so I can attend.
US3	User	As a User, I want to log in to access the features of my role.
US4	Administrator	As Administrator, I want to manage movies to keep the catalog up to date.
US5	Administrator	As an Administrator, I want to create and update schedules to organize functions.
US6	Administrator	As Administrator, I want to manage university sites to define where projections take place.

2.4. CRC Cards

Cuadro 4: CRC Cards - System Classes Summary

Class	Responsibilities	Collaborators
USER	- Register - Login - Read Catalog - Enable reservations	- Pay - Catalog (Movies) - Reservation
USER Attributes	ID, Full name, Email, Number, Password, State	
USER Behaviors	Register(), Login(), Log out(), Modify Profile(), Read Catalog(), Buy Tickets()	
CUSTOMER_USER	- Makes payments	- Pay
ADMIN_USER	- Management system - Manage users	- Pay - Movie - Schedule
MOVIE	- Show info	- Schedule - Ticket - Catalog (Movies)
MOVIE Attributes	ID, Title, Genre, Duration, Rating, Synopsis, Trailers, State, Actors	
MOVIE Behaviors	Show Info()	

Cuadro 5: CRC Cards - Continuation

Class	Responsibilities	Collaborators
HEADQUARTERS	- Show info - Show Movie Theater	- Movie Theater
HEADQUARTERS Attributes	ID, Name, Direction	
HEADQUARTERS Behaviors	Show Info()	
MOVIE THEATER	- Show info - Manage capacity	- Schedule - Headquarters
MOVIE THEATER Attributes	ID, Number, Ability, ID_HEADQUARTERS	
MOVIE THEATER Behaviors	Show seats available()	
SCHEDULE	- Show info	- Movie - Cine Theater - Ticket
SCHEDULE Attributes	ID, Date, Hour, ID_MOVIE, ID_MOVIE_THEATER	
SCHEDULE Behaviors	Show Info()	

Cuadro 6: CRC Cards - Final Part

Class	Responsibilities	Collaborators
TICKET	- Show info	- Movie - Schedule
TICKET Attributes	ID, Price, Seat, ID_SCHEDULE, ID_MOVIE	
TICKET Behaviors	Show info()	
RESERVATION	- Book ticket	- User - Ticket - Payment
RESERVATION Attributes	ID, Reservation_date, ID_USER, ID_TICKET	
RESERVATION Behaviors	Create Reservation(), Delete Reservation()	
PAYMENT	- Make a pay - Verify pay	- USER
PAYMENT Attributes	ID, Cost, Date, Payment Method, ID_RESERVATION, ID_USER	
PAYMENT Behaviors	Show info(), Verify()	

2.5. Class Diagram

Class diagram showing the inheritance structure and relationships between classes, followed by a second class diagram that applies SOLID principles and builds interfaces for reservations, payments, and information display.

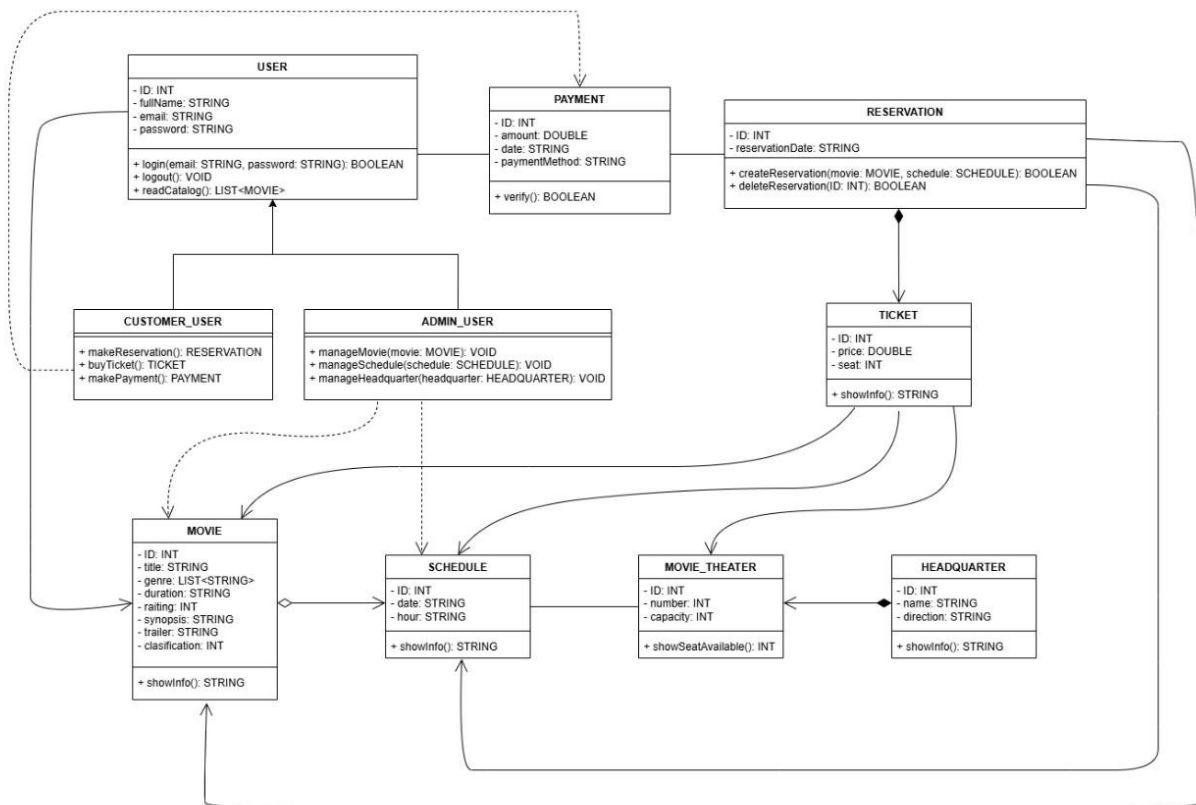


Figura 1: Diagram of Class 1

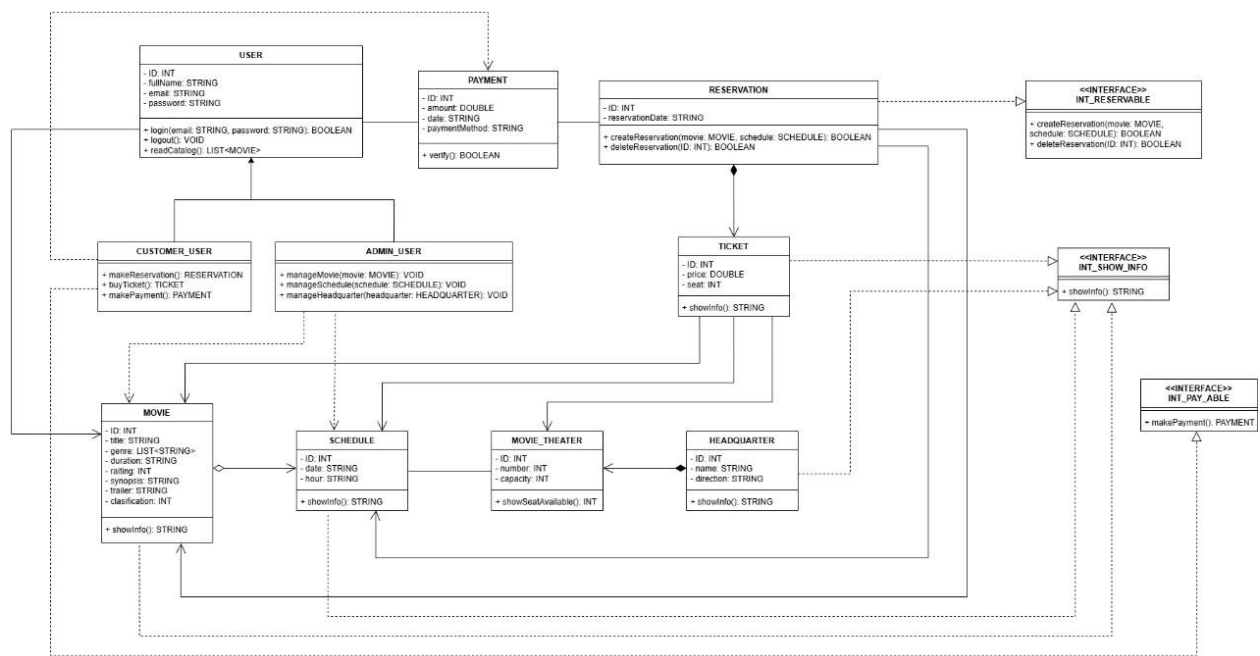


Figura 2: Class Diagram 2 - Applying SOLID Principles

2.6. SOLID Principles Implementation Table

Cuadro 7: Implementation of SOLID Principles in DistriCine

Principle	Acronym	Implementation
Single Responsibility	SRP	Each class has a clear function: USER handles authentication, RESERVATION manages reservations, and PAYMENT controls payments. This prevents overloading the classes and simplifies maintenance.
Open/Closed	OCP	Classes are open to extension (they can inherit or implement new interfaces) but closed to modification. For example, if a new user type or a new payment method is added, it is not necessary to alter the existing code.
Liskov Substitution	LSP	The subclasses (CUSTOMER_USER, ADMIN_USER) can replace the USER class without altering the system's operation, since they maintain the same structure and expected behavior.
Interface Segregation	ISP	Small, specific interfaces (INT_Payable, INT_Reservable, INT_ShowInfo) are defined to prevent classes from implementing methods they don't need. Each class only implements the interfaces that correspond to it.
Dependency Inversion	DIP	Classes rely on abstractions rather than concrete implementations. For example, CUSTOMER_USER makes a payment through the IPayable interface, not directly through the PAYMENT class. This allows the payment method to be changed without affecting the rest of the system.

2.7. Sequence Diagram

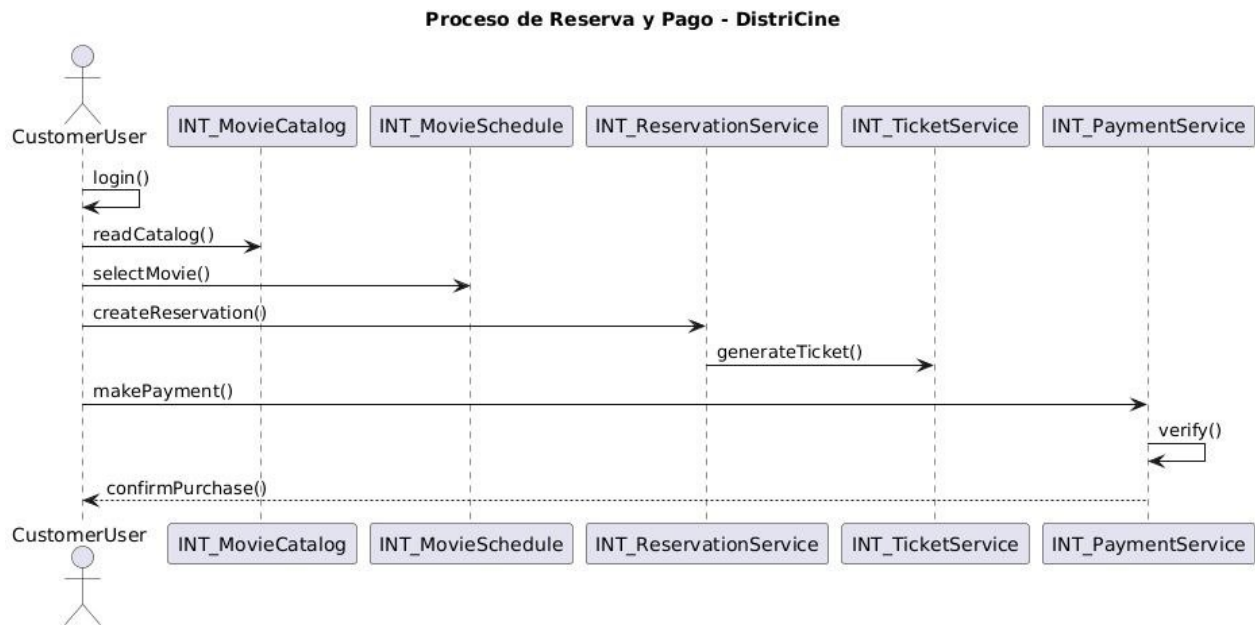


Figura 3: Sequence Diagram

3. Implementation of SOLID Principles in DistriCine

The DistriCine architecture was reorganized to strengthen its maintainability, extensibility, and internal consistency by explicitly applying each of the SOLID principles. The following describes how these principles were integrated into the system design, based on the final UML diagram.

3.1. Single Responsibility Principle (SRP)

Cuadro 8: SRP Implementation Examples

Class	Single Responsibility
USER	Basic authentication and catalog readings only
CUSTOMER_USER	Customer-specific operations (booking, paying, purchasing tickets)
ADMIN_USER	Administrative tasks and system management
MOVIE, SCHEDULE, HEADQUARTER	Store data and present information using showInfo()
RESERVATION	Handle only creation and deletion of reservations

To comply with SRP, classes that combined multiple functions were identified and separated into components with clearly defined responsibilities.

- The USER class was exclusively responsible for basic authentication and catalog readings; customer-specific operations (booking, paying, purchasing tickets) were moved to CUSTOMER_USER, and administrative tasks were delegated to ADMIN_USER.
- Domain entities (MOVIE, SCHEDULE, HEADQUARTER, TICKET, etc.) are limited to storing data and presenting information using showInfo().
- RESERVATION was reduced to handling only the creation and deletion of reservations, avoiding mixing payment or room management logic.

With this, each class has only one valid reason to change.

3.2. Open/Closed Principle (OCP)

The design allows extending functionality without modifying existing classes.

- USER was declared as a generalizable base class. User type variations (CUSTOMER_USER, ADMIN_USER) are achieved through inheritance, allowing adding additional roles (such as STAFF_USER) without touching the parent class.
- The use of the common interface INT_SHOW_INFO allows extending new displayable elements (e.g., PROMOTION, EVENT) without modifying existing classes.
- The payment processor and reservation logic can be extended by adding new interface implementations without altering the core of the system.

This ensures that the system grows without compromising the stability of the already tested code.

3.3. Liskov Substitution Principle (LSP)

Subclasses were designed to behave as valid substitutes for their base classes.

- Any instance of CUSTOMER_USER or ADMIN_USER can be treated as a USER without generating inconsistencies. They all correctly implement the inherited methods (login, readCatalog).
- Classes that implement INT_SHOW_INFO (such as MOVIE, HEADQUARTER, or TICKET) guarantee the uniform availability of the showInfo() method, allowing them to be treated polymorphically.
- Reservation and payment operations respect this principle by relying on stable abstractions (interfaces), ensuring that new implementations do not alter expected behavior.

This maintains consistent and predictable behavior throughout the system.

3.4. Interface Segregation Principle (ISP)

To avoid overly broad interfaces or forcing classes to implement unnecessary methods, small and specific interfaces were designed:

- INT_SHOW_INFO encapsulates only the presentation operation.
- INT_RESERVABLE defines the minimum actions related to reservations.
- INT_PAYABLE groups only the behavior related to payments.

In this way, each class implements only the interfaces it actually needs, maintaining a modular and easily modifiable system.

3.5. Dependency Inversion Principle (DIP)

Dependency was promoted toward abstractions instead of concrete classes.

- Payment operations were abstracted through the INT_PAYABLE interface, allowing multiple payment types without coupling the system to a single implementation.
- RESERVATION, instead of interacting directly with concrete payment or ticket classes, operates on interfaces (INT_RESERVABLE, INT_PAYABLE) that allow replacing or extending behaviors without modifying the internal logic.
- The components responsible for displaying information depend on INT_SHOW_INFO, which eliminates rigid dependencies between classes.

Thanks to this investment, DistriCine is able to incorporate new payment methods, new types of reservations, or new catalog items without affecting the upper layers of the system.

4. Work in Progress Code & Documentation

4.1. S – Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change.

Listing 1: SRP Implementation Example

```
1 public class Movie {  
2     private String title;  
3     private String genre;  
4     private int duration;  
5     private String synopsis;  
6  
7     public void displayInfo() {  
8         System.out.println(title + " (" + genre + ")");  
9     }  
10 }
```

Explanation:

The Movie class is responsible only for managing movie data and showing basic information. It does not handle ticket sales or user management, maintaining a single responsibility.

4.2. O – Open/Closed Principle (OCP)

Definition: Classes should be open for extension but closed for modification.

Listing 2: OCP Implementation Example

```
1 public abstract class User {
2     protected String name;
3     protected String email;
4     public abstract void accessSystem();
5 }
6
7 public class CustomerUser extends User {
8     public void accessSystem() {
9         System.out.println("Customer accessing the movie catalog...")
10    };
11 }
12
13 public class AdminUser extends User {
14     public void accessSystem() {
15         System.out.println("Admin managing movie listings...");
16    }
17 }
```

Explanation:

User can be extended to new roles (e.g., StaffUser) without modifying the base class, fulfilling the Open/Closed Principle.

4.3. L – Liskov Substitution Principle (LSP)

Definition: Subclasses should be substitutable for their base classes.

Listing 3: LSP Implementation Example

```
1 public void showAccess(User user) {
2     user.accessSystem();
3 }
```

Explanation:

Both CustomerUser and AdminUser can replace User without breaking functionality since both override accessSystem(). This respects LSP because the program works correctly regardless of which subclass is used.

4.4. I – Interface Segregation Principle (ISP)

Definition: Clients should not be forced to depend on interfaces they do not use.

Listing 4: ISP Implementation Example

```
1 public interface ReservationActions {
2     void makeReservation();
3 }
4
5 public interface PaymentActions {
6     void makePayment();
7 }
```

Explanation:

By separating actions into different interfaces, a class that only handles reservations does not need to implement payment-related methods. This keeps the system modular and avoids "fat interfaces".

4.5. D – Dependency Inversion Principle (DIP)

Definition: High-level modules should not depend on low-level modules, but both should depend on abstractions.

Listing 5: DIP Implementation Example

```
1 public interface PaymentProcessor {
2     void processPayment(double amount);
3 }
4
5 public class CreditCardPayment implements PaymentProcessor {
6     public void processPayment(double amount) {
7         System.out.println("Processing credit card payment of $" +
8             amount);
9     }
10 }
11
12 public class Reservation {
13     private PaymentProcessor processor;
14
15     public Reservation(PaymentProcessor processor) {
16         this.processor = processor;
17     }
18
19     public void confirm() {
20         processor.processPayment(20.0);
21     }
22 }
```

Explanation:

Reservation depends on the abstraction `PaymentProcessor` rather than a concrete class. This allows changing the payment method (e.g., `PayPal`, `CreditCard`) without altering the reservation logic.

5. Reflection and Conclusions

Working on Workshop 3 of the `DistriCine` project allowed us to understand the importance of applying SOLID principles not just as theoretical rules, but as practical tools for building clean, maintainable, and adaptable software. By separating responsibilities, depending on abstractions, and designing classes that are open to extension but closed to modification, the system becomes more flexible to future changes—something essential in real-world environments like the university setting, where requirements constantly evolve. This approach helped us see object-oriented programming not only as a way to organize code, but as a structured way of thinking about problem-solving.

The application of SOLID principles transformed our perspective on software design. We moved from focusing solely on functionality to considering maintainability, scalability, and code quality. The Single Responsibility Principle taught us the value of focused, cohesive classes. The Open/Closed Principle showed us how to design systems that can grow without breaking existing functionality. The Liskov Substitution Principle reinforced the importance of proper inheritance hierarchies. The Interface Segregation Principle demonstrated the power of focused, client-specific interfaces. Finally, the Dependency Inversion Principle highlighted the benefits of depending on abstractions rather than concrete implementations.

This experience has been invaluable in developing our skills as software engineers. We now recognize that good software design is not just about making code work, but about making it resilient to change, easy to understand, and simple to extend. The SOLID principles provide a framework for achieving these goals, and their application in the `DistriCine` project has given us practical experience that we can carry forward into future software development projects.